

Lab class 2 Exercise Sheet

Model answers

Question 1:

You have a random graph with a mean degree of 4 and a mean excess degree of 4. You then decide to remove all nodes of degree 0 and 1. Do you expect the mean excess degree to go up or down? (*hint: remember the definition of excess degree*)

You do not expect the mean excess degree to go up or down. In a network without degree correlations, nodes of degree 1 and 0 do not contribute to the excess degree at all.

! Important

I somewhat regret having asked that question because a proper demonstration is a bit more involved than can be expected from this module. I will be happy to explain the principle to anyone who asks. The simplest way to demonstrate it is to use so-called *probability generating functions* (PGF). For a tutorial on this, please see Appendix in [1] for some basics and then [2] for its relevance to understanding networks and network robustness in particular. If there is demand, I will write a complete solution.

For now, let's get empirical confirmation using large ER (uncorrelated) networks with a low-enough probability that we guarantee that there will be nodes of degree 0 and degree 1.

```
import networkx as nx
import numpy as np

# Function to calculate mean degree
def mean_degree(G):
    degrees = [deg for _, deg in G.degree()]
    return np.mean(degrees)

# Function to calculate mean excess degree
def mean_excess_degree(G):
    degrees = np.array([deg for _, deg in G.degree()])
```

```

    if np.sum(degrees) == 0:
        return 0 # Avoid division by zero
    return np.sum(degrees * (degrees - 1)) / np.sum(degrees)

# Generate a large Erdős-Rényi (ER) network with low probability
N_large = 2000
p_low = 0.002 # Low probability to ensure nodes with degree 0 and 1

num_experiments = 100
mean_degrees_before = []
mean_excess_degrees_before = []
mean_degrees_after = []
mean_excess_degrees_after = []
nodes_removed_counts = []

for _ in range(num_experiments):
    er_graph_low_p = nx.erdos_renyi_graph(n=N_large, p=p_low)

    # Calculate mean degree and mean excess degree before removing nodes
    er_mean_k_before = mean_degree(er_graph_low_p)
    er_mean_excess_k_before = mean_excess_degree(er_graph_low_p)

    # Remove nodes with degree 0 and 1
    nodes_to_remove = [
        node for node, degree in dict(er_graph_low_p.degree()).items()
        if degree <= 1
    ]
    er_graph_low_p.remove_nodes_from(nodes_to_remove)

    # Calculate mean degree and mean excess degree after removing nodes
    er_mean_k_after = mean_degree(er_graph_low_p)
    er_mean_excess_k_after = mean_excess_degree(er_graph_low_p)

    # Store results
    mean_degrees_before.append(er_mean_k_before)
    mean_excess_degrees_before.append(er_mean_excess_k_before)
    mean_degrees_after.append(er_mean_k_after)
    mean_excess_degrees_after.append(er_mean_excess_k_after)
    nodes_removed_counts.append(len(nodes_to_remove))

# Print results

```

```

print(f"Before removal: \
      Mean Degree = {np.mean(mean_degrees_before):.2f}, \
      Mean Excess Degree = {np.mean(mean_excess_degrees_before):.2f}")
print(f"After removal: \
      Mean Degree = {np.mean(mean_degrees_after):.2f}, \
      Mean Excess Degree = {np.mean(mean_excess_degrees_after):.2f}")
print(f"Average number of nodes removed (degree 0 and 1): \
      {np.mean(nodes_removed_counts):.2f}")

```

```

Before removal:   Mean Degree = 4.00,   Mean Excess Degree = 3.99
After removal:   Mean Degree = 4.24,   Mean Excess Degree = 3.99
Average number of nodes removed (degree 0 and 1):   182.97

```

So for a sufficiently large network, we have mean degree and mean excess degree being identical before removal (see next Question). Once we remove the nodes of degree 0 and 1, the mean degree increases (not surprisingly since we have more nodes with higher degree than before) but the mean excess degree doesn't change, confirming that nodes of degree 0 and 1 did not contribute to the mean excess degree. NB: This is not true in networks with correlations.

Question 2:

Keeping in mind the definition of excess degree, an interesting question is the relationship between mean degree and mean excess degree for a given network. Do you expect it to be generally lower?

Think about it a little bit then write code to generate the following 3 networks:

- Regular network (`_nx.random_regular_graph(d, n)`) with mean degree 4, i.e., all nodes have exactly degree 4.
- Erdos-Renyi random network (`nx.erdos_renyi_graph(n, p)`) also with mean degree 4. We will study this model in more detail very shortly but for now all you need to know is that that in an ER network, the mean degree $\langle k \rangle$ is given by $p(N - 1)$ where p is the probability of a node being connected and N is the size of the network. So set p so that the $\langle k \rangle$ can be expected to be 4.
- Barabasi-Albert random network (`nx.barabasi_albert_graph(n, m)`) also with mean degree 4. We will also study this model in more detail. Here ensuring the mean degree is 4 is a bit tricky. For the purpose of this exercise, simply set the parameter m to 2. This parameter has to be an integer so this will not be exact (see below).

Then, write functions to extract the mean degree and mean excess degrees for all 3 networks. What do you notice regarding the relationship between mean degree and mean excess degree?

If you want to convince yourself, repeat the experiment a number of times to get mean and standard deviation.

NB: Following a number of interactions in lab classes, it has become clear that many of you have simply assumed that calculating the mean excess degree meant taking the excess degree for each node and then averaging over all nodes, in which case, it really doesn't matter what the network looks like. The mean excess degree would always be the mean degree minus 1. The problem with this, however, is that this approach is ignoring a fundamental aspect of the definition of excess degree which is that this is the number of connections of a node excluding the edge **used to reach that node**. This highlighted bit is critical. Your calculation of the mean excess degree must be conditional to following an edge. And now, what you will find is that when you pick an **edge at random**, you are more likely to pick an edge that connects to a hub than an edge that connects to a leaf node. When you look at the mean degree, instead, you pick a **node at random**, and in this case, you are equally likely to pick a hub or a leaf.

The previous question will have already provided some clue that the answer is not automatic. In other words, the fact that excess degree is defined as degree minus 1 (to account for the edge used to reach the node) does not necessarily imply that the mean excess degree will always be less than the average degree. Without doing any simulation, it should be immediately obvious to you that if a network had no heterogeneity in degree, i.e., a regular network where all nodes have the same degree, then, in that case, the mean excess degree would be exactly the mean degree minus 1. But when there is heterogeneity, i.e., some nodes have a higher degree, then, we experience again that sampling bias we discovered when we discussed the friendship paradox in the previous lab class. When following a link at random, you are more likely to land on a neighbour with high degree than one with low degree. It makes sense since there is only one way to reach a neighbour with degree 1 for example whereas there are many ways to reach a hub. So let's generate the 3 different types of networks and see what happens. The functions to calculate mean degree and mean excess degree are identical to the functions used in the previous question.

```
import networkx as nx
import numpy as np

# Number of nodes and target mean degree
N = 100
k_mean = 4

# 1. Generate a regular network (k-regular graph)
regular_graph = nx.random_regular_graph(d=k_mean, n=N)

# Calculate mean degree and mean excess degree
regular_mean_k = mean_degree(regular_graph)
regular_mean_excess_k = mean_excess_degree(regular_graph)
```

```

# 2. Generate an Erdős-Rényi (ER) network with same mean degree
p_er = k_mean / (N - 1) # Probability for ER model
er_graph = nx.erdos_renyi_graph(n=N, p=p_er)

# Calculate mean degree and mean excess degree
er_mean_k = mean_degree(er_graph)
er_mean_excess_k = mean_excess_degree(er_graph)

# 3. Generate a Barabási-Albert (BA) network with same mean degree
m_ba = k_mean // 2 # BA model parameter (must be an integer)
ba_graph = nx.barabasi_albert_graph(n=N, m=m_ba)

# Calculate mean degree and mean excess degree
ba_mean_k = mean_degree(ba_graph)
ba_mean_excess_k = mean_excess_degree(ba_graph)

# Store results
results = {
    "Network Type": ["Regular", "Erdős-Rényi", "Barabási-Albert"],
    "Mean Degree": [regular_mean_k, er_mean_k, ba_mean_k],
    "Mean Excess Degree": [regular_mean_excess_k, er_mean_excess_k, ba_mean_excess_k]
}

import pandas as pd
df = pd.DataFrame(results)
print(df)

```

	Network Type	Mean Degree	Mean Excess Degree
0	Regular	4.00	3.000000
1	Erdős-Rényi	3.94	3.939086
2	Barabási-Albert	3.92	5.831633

So, you should see the expected relationship for the regular network. Numbers should be quite close for ER, and for BA, the mean excess degree should be higher! Now, let's repeat the experiment 100 times to get more robust estimates.

```

# Repeat the experiment multiple times to get mean and standard deviation
num_experiments = 100
results = {
    "Network Type": ["Regular", "Erdős-Rényi", "Barabási-Albert"],
    "Mean Degree": [],

```

```

    "Mean Excess Degree": [],
    "Mean Degree Std": [],
    "Mean Excess Degree Std": []
}

for network_type in results["Network Type"]:
    mean_degrees = []
    mean_excess_degrees = []

    for _ in range(num_experiments):
        if network_type == "Regular":
            G = nx.random_regular_graph(d=k_mean, n=N)
        elif network_type == "Erdős-Rényi":
            G = nx.erdos_renyi_graph(n=N, p=p_er)
        elif network_type == "Barabási-Albert":
            G = nx.barabasi_albert_graph(n=N, m=m_ba)

        mean_degrees.append(mean_degree(G))
        mean_excess_degrees.append(mean_excess_degree(G))

    results["Mean Degree"].append(np.mean(mean_degrees))
    results["Mean Excess Degree"].append(np.mean(mean_excess_degrees))
    results["Mean Degree Std"].append(np.std(mean_degrees))
    results["Mean Excess Degree Std"].append(np.std(mean_excess_degrees))

df = pd.DataFrame(results)
print(df)

```

	Network Type	Mean Degree	Mean Excess Degree	Mean Degree Std \
0	Regular	4.0000	3.000000	0.000000e+00
1	Erdős-Rényi	3.9904	3.928987	2.662552e-01
2	Barabási-Albert	3.9200	6.515000	1.776357e-15

	Mean Excess Degree Std
0	0.000000
1	0.304043
2	0.715721

Now you should notice that unlike what we discussed in the first question, the mean excess degree is not exactly equal to the mean degree for the ER network. This is because the relationship only holds in the limit of large networks. So let's ramp up the number of nodes just for the ER network:

```

num_experiments = 100
N_large = 2000

results_large = {
    "Mean Degree": [],
    "Mean Excess Degree": [],
    "Mean Degree Std": [],
    "Mean Excess Degree Std": []
}

mean_degrees = []
mean_excess_degrees = []

p_er_large = k_mean / (N_large - 1) # Update probability for new size

for _ in range(num_experiments):
    G = nx.erdos_renyi_graph(n=N_large, p=p_er_large)

    mean_degrees.append(mean_degree(G))
    mean_excess_degrees.append(mean_excess_degree(G))

results_large["Mean Degree"].append(np.mean(mean_degrees))
results_large["Mean Excess Degree"].append(np.mean(mean_excess_degrees))
results_large["Mean Degree Std"].append(np.std(mean_degrees))
results_large["Mean Excess Degree Std"].append(np.std(mean_excess_degrees))

df_large = pd.DataFrame(results_large)
print(df_large)

```

	Mean Degree	Mean Excess Degree	Mean Degree Std	Mean Excess Degree Std
0	4.01852	4.018691	0.063216	0.070284

and now it should be pretty close...

References

1. J. C. Miller, A primer on the use of probability generating functions in infectious disease modeling. (2024). <https://doi.org/10.48550/arXiv.1803.05136>.
2. T. Gross & L. Barth, Network Robustness Revisited. *Frontiers in Physics*, **10** (2022). <https://doi.org/10.3389/fphy.2022.823564>.